

กรณีศึกษาที่ 6: ระบบสร้างคู่เพื่อนร่วมกิจกรรม

นักศึกษาจำนวน (n) คนต้องจับคู่กันทุกคู่ เพื่อทำกิจกรรม

แนะนำตัว โดยนักศึกษาคนหนึ่งต้องจับคู่กับ นักศึกษาทุกคนที่อยู่
คู่ต่างจากตนเอง ตัวอย่าง หากมีนักศึกษา A, B, C และ D คู่ที่เกิดขึ้น

คือ A-B

A-C

A-D

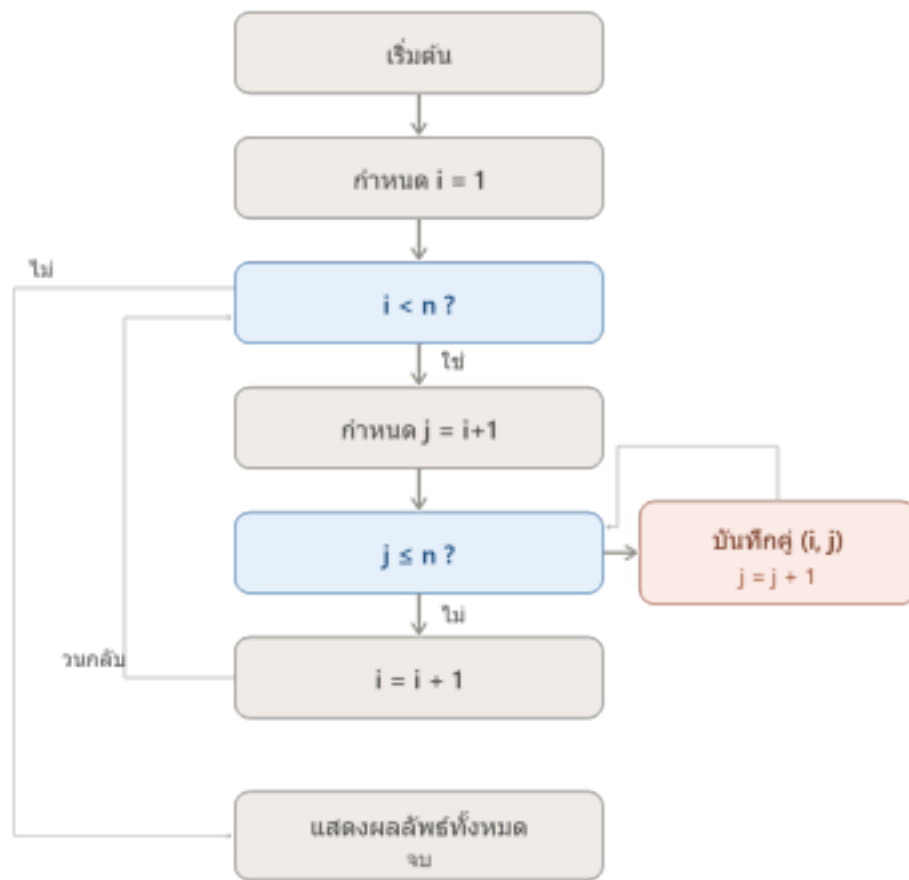
B-C

B-D

C-D

งานที่มอบหมาย

1. ออกแบบบล็อกกรณีสร้างคู่ที่ไม่ซ้ำกัน



สีน้ำเงิน = จุดตัดสินใจ · สีส้ม = ขั้นตอนบันทึกคู่

2. เขียนโปรแกรม Java

```

import java.util.ArrayList;

public class ActivityPartnerPairingSystem {
    public static void main(String[] args) {
        // กำหนดรายชื่อนักศึกษาตามกรณีศึกษา (มี 4 คน)
        String[] students = {"A", "B", "C", "D"};
        // n คือจำนวนนักศึกษาทั้งหมด
        int n = students.length;
    }
}

```

```

// สร้าง ArrayList สำหรับเก็บผลลัพธ์การจับคู่ทั้งหมด
ArrayList<String> recordedPairs = new
ArrayList<>();

// เริ่มต้นตัวแทนนักศึกษาคนแรก (กำหนดให้เป็นลำดับที่ i)
int i = 1;

// วงวนเพื่อให้นักศึกษาคนที่ i จับคู่กับคนที่อยู่ถัดไปจนครบ
while (i < n) {

    // กำหนดนักศึกษาคนที่สอง (ลำดับที่ j) ให้เริ่มต้นที่คน
    ถัดจาก i เสมอเพื่อไม่ให้จับคู่ซ้ำคนเดิม
    int j = i + 1;

    // วงวนให้นักศึกษาคนที่ j รันไปจนถึงนักศึกษาคนสุดท้าย
    while (j <= n) {

        // ดึงชื่อนักศึกษาจาก Array
        String student1 = students[i - 1];
        String student2 = students[j - 1];

        // นำชื่อมาต่อกันให้อยู่ในรูปแบบที่โจทย์ต้องการ
        recordedPairs.add(student1 + "-" +
student2);

        // ขยับลำดับ j ไปยังนักศึกษาคนถัดไป
        j = j + 1;
    }
}

```

```

        // เมื่อนักศึกษาคนที่ i จับคู่กับทุกคนครบแล้ว ให้ขยับ
ลำดับ i เป็นคนถัดไป
        i = i + 1;
    }

    // นำผลลัพธ์คู่ทั้งหมดที่บันทึกไว้ใน List มาแสดงผลทางหน้าจอ
    System.out.println("Result pairs are:");
    for (String pair : recordedPairs) {
        System.out.println(pair);
    }
}
}

```

```

Result pairs are:
A-B
A-C
A-D
B-C
B-D
C-D

```

3. หาจำนวนคู่ทั้งหมด ในรูปฟังก์ชันของ (??)

จำนวนคู่ทั้งหมด (??(??)) สามารถเขียนในรูปฟังก์ชัน

องจำนวน สมาชิก ?? ได้ดังนี้ ??(??) = ??(??-1)

คำอธิบาย/ ที่มาของฟังก์ชัน:

1. การจับคู่สมาชิก $\diamond \diamond$ ตัว โดยไม่ซ้ำคู่เดิม และไม่จับคู่กับตัวเอง
เป็นการเลือกสมาชิก 2 ตัวจาก $\diamond \diamond$ ตัว (การจับคู่หรือ Combination)

$\diamond \diamond$

(-2)

เขียนได้เป็น จำนวนจากสูตร:

$$\diamond \diamond (\diamond \diamond) = \diamond \diamond !$$

$$2! (\diamond \diamond - 2)! = \frac{\diamond \diamond (\diamond \diamond - 1)}{2}$$

2. ตัวอย่าง การทดสอบ:

หากมีสมาชิก $\diamond \diamond = 4$ ตัว (A, B, C, D)

$$\diamond \diamond (4) = 4(4-1)$$

$$_2 = 6$$

ซึ่งตรงกับผลลัพธ์ในระบบ ได้แก่ A-B, A-C, A-D, B-C, B-D

และ C-D

4.วิเคราะห์ Time Complexity

กระบวนการวิเคราะห์:

’กนั 2 ซั้ (Nested

1. พิจารณาโครงสร้างลป :โปรแกรมไขลั ปซุ วน

Loops)

○ลป นอก(i) : ทำงานตั้งแตั i=0ถึง n - 2 (รวม
ทั้งหมดั n - 1 รอบ)

○ลป ใน (j) : ทำงานเริ่มจากj=i+1ถึง n - 1

2. นับจำนวนรอบการทำงานรวม $T(n)$:

○รอบที่ i=0 :ลปใน ทำงาน n - 1ครั้

○รอบที่ i=1 :ลปใน ทำงาน n - 2ครั้

○...

○รอบสุดท้าย:ลปใน ทำงาน 1ครั้

นำมารวมกันั เป็นอนุกรม :

$$T(n) = (n - 1) + (n - 2) + \dots + 2 + 1$$

$$T(n) = \frac{n(n-1)}{2} = \frac{1}{2}n^2 - \frac{1}{2}n$$

พิจารณาอัตราการเติบโต: เมื่อ n มีค่าเพิ่มขึ้น
พจน์ $\frac{1}{2}n^2$ จะส่งผลต่อเวลา มากที่สุด ดังนั้นเวลา
ในการทำงานจะเติบโตแบบยกกำลังสอง (n^2)

5.วิเคราะห์พื้นที่หน่วยความจำ เมื่อ

แสดงผลทันที: Space Complexity = $O(1)$ เพราะไม่เก็บข้อมูล
ไว้ในหน่วย

ความจำ ใช้เพียงตัวแปร สำหรับควบคุมการวนลูป

เก็บคู่ทั้งหมดไว้ใน ArrayList: Space Complexity = $O(n^2)$ เพราะต่อ

งเก็บคู่ทั้งหมด จำนวน $\frac{n(n-1)}{2}$ คู่ซึ่งเพิ่มขึ้น

เป็นกำลังสองตามจำนวนของนักศึกษา

สรุป: ผลลัพธ์เมื่อสร้างคู่ (A, B) ได้ก็พิมพ์ออก

มาเลยแต่ไม่ได้

เก็บค่าเอาไว้

ส่วน array list จะเก็บทุกคู่ใน array list